

Зачем роботу обучаемая модель

M18 • 30.04.2028 - 04.06.2028 • 75 часов

Результат месяца

Создай воспроизводимый классификатор контакта стопы с поверхностью и сравни его с простым правилом и логистической регрессией. На вход подавай доступные роботу измерения суставов и IMU, а также признаки движения ноги. На выходе получай вероятность контакта, решение «контакт», «нет контакта» или «недостаточно данных» и время его получения. Сохрани данные, состав выборок, конфигурацию обучения, модель, разбор ошибок и измерения времени выполнения на CPU.

Обучение здесь решает узкую задачу восприятия. Классификатор помогает оценивать, какой стопе можно доверять при вычислении состояния корпуса. Он пока работает по записям или рядом с классическим алгоритмом в симуляции. Ошибочное подтверждение контакта может заставить оцениватель считать скользящую или поднятую ногу неподвижной; поэтому средняя точность сама по себе не определяет пригодность модели.

Что должно быть готово до начала

Подготовь синхронизированные записи предыдущих симуляций: время, идентификаторы эпизода и ноги, положения и скорости суставов, угловую скорость корпуса, положение или скорость стопы и доступную оценку усилия. Эталонный контакт симулятора сохраняй только для разметки и проверки. Если записей мало, выполни контролируемые движения одной ноги: свободный перенос, касание, устойчивую опору, отрыв и скольжение. Сложная походка для сбора данных не требуется.

Проверь, что умеешь читать CSV и Parquet, строить временной график, объяснять среднее и стандартное отклонение, вычислять скалярное произведение и производную квадратичной ошибки. Недостающие навыки отработай в первом блоке. Опыт разработки используй для описания формата данных, воспроизводимости и поиска причин ошибок модели; повторять все основы Python не нужно.

Что подтверждает результат

Главный результат — обоснованный учебный эксперимент. Сеть пока не допускается к управлению физическим роботом. Исходный учебный план упоминает G3, а общий календарь отводит интеграционным проверкам отдельное время. В M18 подготовь данные, модель и описание ограничений для этого этапа. До любого физического применения требуются проверки независимо от даты G3. В следующих модулях классические оцениватель и контроллер должны работать и без машинного обучения.

Связь с рукой прямая: тот же процесс применим к оценке контакта пальца с предметом. Однако положительная метка «касание» не равна «надёжный захват». В этом месяце используй одну основную задачу на стопе, а перенос на палец оформи как описание различий признаков и ошибок, без второго обязательного обучения.

75 часов: работа от данных к решению

Четыре учебных блока

Часы 1–20: 10 часов теории и анализа плюс 10 часов практики. Разбери регрессию, классификацию, функцию потерь, смещение и разброс. Определи признаки, разметку и состав обучающей, валидационной и тестовой выборки из целых эпизодов. Подготовь описание набора данных, проверь синхронизацию и зафиксируй первый классический базовый метод. Результат — данные, в которых правильный ответ не попадает во вход модели.

Часы 21–40: 10 часов теории и 10 часов практики. Изучи тензоры, автоматическое дифференцирование, слои, оптимизатор, нормализацию и контроль переобучения. Напиши небольшой обучающий цикл PyTorch и запусти компактную сеть. Результат — воспроизводимый запуск, а не длительный поиск архитектуры. В каждом эксперименте меняется одна гипотеза; исходная конфигурация сохраняется неизменной.

Часы 41–60: 10 часов теории и 10 часов практики. Разбери матрицу ошибок, метрики precision и recall, кривые ROC и PR, калибровку, дисбаланс классов и сдвиг данных. Исследуй неудачи, исключай группы признаков и проверь отказ от ответа. Порог подтверждения контакта выбирай по валидационной выборке; тестовые эпизоды оставь закрытыми до итогового сравнения.

Часы 61–75: 5 часов теории и анализа плюс 10 часов практики. Разбери экспорт, численную точность, квантование и различие задержки с пропускной способностью. Измерь полный путь от готового измерения до решения, воспроизведи модель в отдельном процессе и подготовь решение о вычислительной платформе. Итого — 35 часов теории и 40 практики; самостоятельное ожидание обучения не заменяет учебную работу.

Как размещать занятия

Каждые 10 часов теории раздели на пять двухчасовых занятий. Практику выполняй воскресными блоками по 10 часов; каждый можно разбить на четыре отрезка с перерывами, которые в учебное время не входят. Последние 5 часов анализа распредели как 2 + 2 + 1. Нумерация означает порядок учебной нагрузки. Граничные даты могут совпадать с соседними модулями; отдых, отпуска и интеграционные блоки сохраняются по учебному плану, один час дважды не учитывается.

В начале занятия запиши один проверяемый вопрос: например, «уменьшает ли скорость стопы ложные контакты при отрыве?». В конце сохрани конфигурацию, график, вывод и следующий шаг. Ограничь активный список тремя задачами: текущая гипотеза, известная ошибка данных, ближайшая проверка. Это удержит модуль в 75 часах и поможет отличать улучшение модели от изменения самого эксперимента.

Данные, метки и разделение без утечки

Задача 1. Описание данных о контакте

Для каждого поля укажи имя, единицы, частоту, источник, задержку и доступность на будущем роботе. Раздели таблицы `features` и `ground_truth`. В признаки не должны попадать истинная контактная сила, идентификатор сценария, будущий кадр или координата стопы относительно точной поверхности, если соответствующего датчика на роботе нет. Так ты отделишь измеряемые величины от сведений, известных только симулятору. Сохрани `data_contract.md` и десять строк, проверенных вручную.

Задай положительную метку по контакту симулятора и минимальной нагрузке, выбранной для данной модели. Пограничные моменты касания пометь `uncertain` и опиши правила их оценки. Порог разметки не подстраивай под результат сети. Вручную проверь несколько участков до, во время и после контакта: метка должна соответствовать событию без сдвига на один пакет. Число неопределённых строк укажи отдельно.

Задача 2. Разделение по целым опытам

Сначала присвой `episode_id` целым независимым запускам. Затем составь явный перечень эпизодов для каталогов `train`, `validation` и `test`: например, 60/20/20 процентов эпизодов; число независимых групп важнее точной доли. Соседние окна одного опыта должны оставаться в одной части. Последовательные файлы одной записи робота объедини в группу. Отдельно сохрани набор OOD с новым трением, шумом или нагрузкой.

Проверь отсутствие пересечений идентификаторов и диапазонов исходных записей. Параметры нормализации, заполнения пропусков, балансировки и отбора признаков определяй только по обучающей выборке. При малом числе эпизодов используй в ней групповую перекрёстную проверку, сохраняя отдельные финальные тестовые группы. Покажи распределение классов и условий по частям. Случайное перемешивание строк временного ряда делает проверку неоправданно лёгкой.

Задача 3. Проверка причинной доступности

Для окна длиной W используй лишь измерения с временем не позднее момента решения. Сформируй среднее, разброс и максимум скорости за прошедшее окно, а также флаг недостающих пакетов. Измени будущие значения в исходном файле: прогноз для текущего момента обязан остаться прежним. Результат — тест причинности и график задержки окна. Если для хорошей метрики требуется будущая информация, такой вариант годится для анализа записи, но не для онлайн-оценки контакта.

Начни с 6–12 понятных признаков. Миллионы почти одинаковых кадров не добавляют новых условий. Несколько эпизодов с разными поверхностями и фазами движения полезнее ещё одного часа записи неподвижной стопы.

Базовые модели и функции потерь

Что изучить перед нейросетью

Регрессия предсказывает число, например нормальную нагрузку в ньютонах; классификация — категорию или вероятность контакта. Для учебного сравнения нарисуй зависимость квадратичной ошибки от числового прогноза и бинарной кросс-энтропии от вероятности правильного класса. Не смешивай разные цели: малая ошибка силы не гарантирует надёжного обнаружения касания, особенно около нуля.

В бинарной задаче y принимает 0 или 1, а p обозначает вероятность контакта. Функция потерь равна $-[y \log(p) + (1-y) \log(1-p)]$. Она сильнее штрафует уверенный неверный ответ. Используй численно устойчивую реализацию для логитов; избегай ручного вычисления логарифма нуля и повторного применения sigmoid. Полученная вероятность служит для анализа, а не множителем команд приводам.

Задача 4. Два базовых метода для сравнения

Построй простое правило по доступным признакам: малая скорость стопы вместе с достаточной оценкой нагрузки, если она есть. Пороги выбирай по валидационной выборке. Если нагрузка не измеряется, укажи это и используй кинематику и фазу команды. Второй базовый метод — логистическая регрессия с нормализацией, рассчитанной на обучающей выборке. Оба метода и сеть должны использовать одинаковые эпизоды, входы и задержку окна.

Сохрани `predictions_baseline.csv`, матрицу ошибок и ошибки по эпизодам. Проверь свободно висящую неподвижную ногу: низкая скорость сама по себе не означает контакт. Проверь скользящую опору: контакт может существовать при ненулевой скорости. Эти два случая объясняют, зачем нужен набор признаков и почему даже разумное правило не является физическим доказательством опоры.

Задача 5. Смещение, разброс, регуляризация

Обучи логистическую регрессию на малой и полной обучающих выборках, затем компактную сеть с регуляризацией и без неё. Сравни потери на обучающих и валидационных данных. Высокая ошибка в обеих частях указывает на возможные проблемы признаков, модели или меток. Низкая обучающая ошибка при высокой валидационной требует проверки переобучения и различия условий. Эти наблюдения помогают искать причину, но не устанавливают её автоматически.

После первого базового результата проверь максимум три настройки: например, два значения регуляризации и одну ширину скрытого слоя. Установи общий предел эпох и раннюю остановку по валидационной выборке. Сохрани таблицу с гипотезой, изменением, наблюдением и решением. Вывод об улучшении требует большего, чем один удачный запуск. Окончательную тестовую выборку нельзя многократно использовать для выбора архитектуры.

PyTorch: цикл обучения небольшой модели

Минимальный набор разделов

Разбери форму тензора: строка соответствует окну измерений, столбец — признаку. Для N примеров с D признаками вход имеет форму $N \times D$; выход бинарной модели содержит один логит на пример. Проверь dtype, устройство и размерности целевых значений target. Используй нормализацию и небольшой многослойный перцептрон MLP с одним или двумя скрытыми слоями. Сложные свёртки, трансформеры и поиск архитектуры для этой задачи не нужны.

Задача 6. Один воспроизводимый запуск

Создай Dataset по перечню эпизодов, DataLoader, модель, функцию потерь, оптимизатор и отдельную функцию evaluate. В обучении последовательно обнуляй градиенты, выполняй прямой проход, рассчитывай потери, выполняй обратный проход и шаг оптимизатора. Для оценки отключай накопление градиентов и включай режим eval. Вместе с весами сохраняй нормализацию, порядок признаков и версию схемы. При несовпадении схемы входа загрузка должна явно отклоняться.

Проверь цикл на небольшом наборе правильно размеченных примеров: модель должна существенно уменьшить ошибку обучения. Это проверка работоспособности, а не обобщения. Затем восстанови штатное разделение данных. Намеренно поменяй местами два столбца и проверь, что схема выявляет ошибку до прогноза: ухудшение ответов само по себе слишком поздно указывает на смену формата.

Задача 7. Градиенты и оптимизаторы

Для модели с одним линейным слоем сравни компоненту градиента autograd с центральной конечной разностью при малом изменении веса. Проверь направление обновления и изменение функции потерь после малого шага. Затем сравни SGD и Adam при одинаковом числе эпох, размере пакета и числе примеров. Запиши скорость уменьшения потерь и итоговую валидационную метрику; результат этого опыта не устанавливает универсального преимущества оптимизатора.

Изучи скорость обучения, штраф на веса и простое расписание уменьшения шага. Выполни один опыт с заведомо большим шагом для диагностики расходимости, затем верни рабочую настройку. Для каждого запуска сохраняй начальное значение генератора случайных чисел seed, версии библиотек, устройство, коммит, перечень эпизодов, настройки оптимизатора и лучшие веса. Seed улучшает повторяемость, но не гарантирует побитового совпадения на разных платформах и версиях; сравнивай в закреплённой среде.

Базовая платформа — CPU. Apple Silicon может использовать MPS при поддержке нужных операций, но MacBook M4 не является CUDA-платформой. Для этой небольшой табличной сети покупка GPU не предполагается; сначала измерь фактическое время подготовки данных и обучения.

Качество, уверенность и неизвестные условия

Метрика должна отражать ошибку робота

Пусть положительный класс — контакт. $\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$ показывает, какая доля заявленных контактов верна; $\text{recall} = \text{TP}/(\text{TP}+\text{FN})$ — какая доля настоящих контактов найдена. Ложный контакт FP особенно опасен для контактной одометрии, а пропущенный контакт FN уменьшает доступную информацию. Укажи правило обработки нулевого знаменателя. Дополнительно оцени задержку обнаружения касания и отрыва относительно размеченного события в миллисекундах.

Задача 8. Порог и дисбаланс

На валидационной выборке построй матрицу ошибок, PR-кривую и ROC-кривую. Выбери порог с учётом допустимого числа ложных контактов в учебном сценарии. Этот критерий задай до финального теста; он не является универсальным пределом безопасности. Покажи число событий и примеров каждого класса. Если в записи преобладает опора, правило «всегда контакт» может давать высокую долю верных ответов ассигасу: добавь его, чтобы показать ограниченность этой метрики.

Для редкого отрыва сравни обычную функцию потерь с взвешенной; веса классов рассчитай по обучающей выборке. После изменения способа отбора примеров или весов заново проверь калибровку. Распределение тестовой выборки должно соответствовать заявленному сценарию, без искусственной балансировки. Покажи качество каждого эпизода и среднее по эпизодам, чтобы длинная запись не скрывала полного отказа на короткой.

Задача 9. Калибровка и отказ от ответа

Разбей вероятности на валидационных данных по интервалам. Сравни среднюю предсказанную вероятность с фактической долей контактов и покажи число примеров в интервале: получится диаграмма калибровки (reliability diagram). Рассчитай Brier score как среднее $(p-y)^2$. Для подгонки калибровки выдели отдельные группы из обучающих или валидационных данных; тестовые данные не используй. При малом наборе укажи неопределённость оценки и объём наблюдений.

Определи причины отказа от ответа: отсутствие обязательных полей, устаревшее окно, выход значений за известные диапазоны и недостаточная уверенность. Проверка диапазонов обнаруживает только часть случаев OOD; высокая вероятность сети не доказывает, что условия знакомы. На новом трении и шуме измерь частоту отказов и оставшиеся ошибки. При отказе передавай unknown классическому оценщику, который применяет проверенные правила и не принимает контакт автоматически.

Лабораторная работа: распознавание контакта

Техническое задание `contact_model_v1`

Воспроизводи записи ноги по временным меткам. Сначала проверь схему и актуальность данных, затем сформируй признаки только из доступных к этому моменту измерений. Модель выдаёт вероятность, пороговый блок — статус, отдельная проверка сопоставляет его с эталонным контактом. Команды приводам здесь не отправляются. На одном графике покажи эталонный контакт, прогноз, `unknown`, скорость стопы и причину отказа; линиями отметь касание, отрыв и скольжение.

В описании входных данных укажи файлы, хеши, модель робота, симулятор, длительности, классы и условия. В конфигурации задай окно, признаки, `seed`, модель, предел эпох, пороги и правила отказа. Сохрани прогноз для каждого допустимого момента, метрики эпизодов и измерения времени. Отклонённые строки тоже учитывай: число, причины и доля среди всех измерений должны оставаться видимыми, иначе удаление сложных случаев искусственно улучшит метрику.

Задача 10. Итоговое сравнение на тестовой выборке

После выбора конфигурации зафиксируй коммит и открой тестовые группы. Одинаковым способом проверь правило, логистическую регрессию и выбранный MLP. Сравни `precision`, `recall`, задержку событий, долю `unknown` и время обработки. Парное сравнение проводи на одинаковых эпизодах. Для оценки разброса повторно выбирай целые эпизоды, а не соседние кадры. Если эпизодов недостаточно для уверенного вывода, укажи это.

Критерий учебного плана — улучшение простого базового метода на отложенных данных. До тестирования выбери основную метрику и ограничение на ложные контакты. Если сеть не выигрывает, сохрани отрицательный результат и отметь невыполненный критерий; новые данные собирай в следующей итерации. Воспроизводимая оценка всё равно полезна: она может обоснованно показать, что следует выбрать более простую модель.

Задача 11. Абляция признаков

По очереди исключи сведения о фазе команды, кинематическую скорость и признаки IMU. Каждый вариант обучи на той же обучающей выборке, выбирая настройки по той же валидационной. Для анализа используй отдельный набор, сохраняя независимость финального теста. Сравни ошибки при отрыве, скольжении и неподвижной ноге в воздухе. Сформулируй гипотезу о полезности источника информации и сохрани два графика неудачных эпизодов.

В конце повтори работу из чистого каталога по README. Обучение, оценку качества и измерение времени запускай отдельными командами, чтобы проверка сохранённой модели не требовала повторного обучения.

Испытания: восемь способов ошибиться

Проверки данных и поведения

- 1. Соседние окна в разных выборках.** Подготовь намеренно неверный перечень, где один `episode_id` входит в `train` и `test`. Проверка должна отклонить его до обучения. На диагностических данных сравни случайное разделение строк с групповым. Объясни завышенную оценку корреляцией соседних данных. Сохрани автоматический отчёт о пересечениях и две оценки с указанием способа разделения.
- 2. Неподвижная нога в воздухе.** Воспроизведи остановленный перенос без касания. Правило только по скорости может дать FP; сеть и комбинированный базовый метод проверь в тех же условиях. Запиши длительность ложного контакта и объясни, по каким измерениям случаи можно различить. Если датчиков для этого недостаточно, используй `unknown`.
- 3. Скользящая опора.** Используй контакт с ненулевой тангенциальной скоростью. Метка контакта остаётся положительной, а «нескользкая опора» — другая задача. Проверь, что анализ не называет все скользящие кадры ошибочной разметкой. Для будущего оценщика передай отдельный индикатор качества опоры, не подменяй им текущую цель.
- 4. Пропуск и задержка.** Удали последовательность пакетов и задержи один канал. Запиши давность самого старого использованного измерения. После заданного в конфигурации предела решение должно стать `unknown`. Проверь моменты до и после порога и восстановление после накопления полного окна из уже поступивших данных.

Проверки модели и исполнения

- 5. Новые нагрузка и трение.** Проверь физически корректные OOD-эпизоды за пределами диапазонов обучающей выборки. Сравни ошибки с обычной тестовой выборкой и измерь долю обнаруженных случаев OOD. Опиши уверенный ошибочный ответ и оставшийся риск: идеального обнаружения новых условий ожидать нельзя.
- 6. NaN и нарушенная схема.** Подай нечисловое значение, переставленные колонки и другую единицу скорости. NaN и версия схемы должны отклоняться до сети. Ошибку единиц проверяй явной метаданной и диапазонами: не все изменения единиц обнаружимы по одному числу. Запиши точный код причины.
- 7. Повторяемость.** Повтори фиксированную конфигурацию в той же среде, затем сравни три значения `seed`. Укажи разброс основной метрики и время обучения, включив все запуски. Межплатформенное совпадение оценивай с заранее выбранной численной погрешностью.
- 8. Экспорт и медленный потребитель.** Сравни исходную модель с сохранённой или экспортированной на одинаковых входах, затем искусственно замедли обработку. Позднее чтение не должно менять давность решения. Измерь число нарушенных сроков обработки и реакцию наблюдателя; сохрани фактическую последовательность временных меток.

Последовательность занятий: часы 1-40

Блок 1: задача и данные

Часы 1–2: нарисуй цепочку «движение ноги → измерение → признак → метка → решение». Разбери, чем контакт отличается от устойчивой опоры и от разрешения переноса веса. Выпиши ошибки, которые должен выявить эксперимент. Часы 3–4: разбери регрессию, классификацию, функцию потерь и переобучение; вручную посчитай две потери и матрицу ошибок для десяти прогнозов. Не переходи к PyTorch, пока значения в этой таблице непонятны.

Часы 5–6: изучи групповое разбиение и причинные окна. На бумаге раздели шесть коротких опытов так, чтобы один сценарий не распался на соседние строки разных наборов. Часы 7–8: составь описание признаков, единицы и доступность датчиков. Часы 9–10: зафиксируй базовые модели, главную метрику и тестовый протокол. Эти пять занятий дают необходимые 10 часов анализа.

Часы 11–20 — первая лаборатория: 2 часа на извлечение и ручную проверку меток; 2 на перечни эпизодов и проверку утечки; 2 на причинную подготовку признаков; 2 на правило и логистическую регрессию; 2 на графики неудач и фиксацию базового результата. При трудностях импорта сократи число сценариев, сохранив классы и независимые эпизоды. Нормализацию рассчитывай после разделения данных и сохрани тестовую выборку.

Блок 2: обучение небольшой модели

Часы 21–22: тензоры, формы и типы; таблица размеров от входа до функции потерь. Часы 23–24: autograd и градиентный шаг; ручная проверка производной и конечной разности. Часы 25–26: Dataset, DataLoader, режимы train/eval и отключение градиентов при оценке. Часы 27–28: SGD, Adam, скорость обучения, регуляризация и ранняя остановка. Часы 29–30: сохранение модели, конфигурации, значений seed и журнала эксперимента.

Часы 31–40 — вторая лаборатория: 2 часа на цикл обучения и проверку крошечного набора; 2 на первый полный запуск; 2 на отладку градиентов и подготовки данных; 2 на два ограниченных сравнения гипотез; 2 на восстановление сохранённых весов и воспроизводимость. Сохраняй неудачные запуски. Если сеть не обучается, проверь метки, размерности, градиенты и нормализацию, затем меняй размер модели.

К концу 40-го часа сохрани исходный классический метод без изменений и одну работающую конфигурацию обучаемой модели. Достаточно каталога runs, метрик в CSV и README с командой повторения. Каждое число должно быть связано с конкретными данными и настройками; дополнительный интерфейс, облачный журнал и сложная автоматизация необязательны.

Последовательность занятий: часы 41–75

Блок 3: проверка гипотез

Часы 41–42: precision, recall и PR; ручная проверка знаменателей и смысла положительного класса. Часы 43–44: сравнение ROC и PR при дисбалансе классов, выбор порога по валидационным данным. Часы 45–46: калибровка и её диаграмма, примеры уверенных ошибочных ответов. Часы 47–48: новые условия OOD — шум, нагрузка, трение и временные сдвиги. Часы 49–50: план исключения признаков и перечень ошибок.

Часы 51–60 — третья лаборатория: 2 часа на матрицы ошибок и пороги; 2 на калибровку и unknown; 2 на пропуски и сдвиги; 2 на ограниченное исключение признаков; 2 на окончательное сравнение и разбор двух худших эпизодов. Для каждого числа укажи выборку. После открытия финального теста новая настройка требует объявленной следующей итерации и новой независимой проверки.

Блок 4: измерения при применении модели

Часы 61–62: изучи экспорт и различие формата весов и исполняемого графа; проверь, что весь препроцессор переносится вместе с моделью. Часы 63–64: разберись с размером пакета, задержкой одного решения, прогревом и памятью. Час 65: опиши, какие ограничения применения остаются и что потребуется для последующей G3-проверки. Это последние 5 часов теории, разбитые на 2 + 2 + 1.

Часы 66–75 — четвёртая лаборатория: 2 часа на загрузку, экспорт и сравнение выходов; 2 на измерения CPU; 2 на воспроизведение всего пути от измерения до решения; 2 на повтор из чистого каталога; 2 на отчёт и выбор вычислительной платформы. Измеряй после прогрева, отдельно от холодного старта. Укажи число повторов, batch=1, число потоков, модель CPU, режим питания и конкурирующую нагрузку.

Задача 12. Нужен ли ускоритель

Раздели полную задержку на накопление окна, подготовку, вычисление прогноза и передачу результата. Измерь p50, p95 и p99 — значения, ниже которых лежат 50, 95 и 99 процентов наблюдений, — а также максимум и число просрочек. Эти измерения не доказывают верхнюю границу времени. Пропускная способность в примерах/с при большом пакете не заменяет задержку одного свежего измерения. GPU может ускорить сеть, добавив при этом расходы передачи.

Обоснуй необходимость Jetson и CUDA по задержке, памяти, энергопотреблению и доступным операциям. Покупка и допуск обучаемого управления ходьбой рассматриваются на отдельном G3. Если CPU укладывается в требования, покупать ускоритель не нужно. В противном случае сначала найди причину задержки и сократи модель. Квантование проверяй отдельно: повтори оценку точности, калибровки и OOD. Сокращение памяти не гарантирует ускорения.

Приёмка и передача результатов дальше

Материалы, подтверждающие результат

Создай каталог `contact_model_v1` с `README`, `data_contract`, `manifests`, `configs`, `baseline`, `model`, `reports` и `measurements`. В `README` укажи способ получения исходных логов и команды `train`, `evaluate`, `replay`, `profile`. В отчёте должны быть цель, измеряемые признаки, происхождение меток, границы применения, версии среды и характеристики устройства. Храни точные конфигурации даже для отклонённых моделей: иначе невозможно понять, почему финальный выбор оказался лучше.

Критерий 1: нет пересечения эпизодов, будущих кадров и настройки обработки на тестовых данных; приложены автоматические и ручные проверки. Критерий 2: базовый метод сравнен с сетью на закрытой до финала выборке по заранее выбранной метрике и ограничению FP; при отсутствии улучшения критерий остаётся невыполненным. Критерий 3: повтор с теми же `config` и `seed` даёт объяснимо близкий результат в одной среде. Критерий 4: измерен весь путь данных, включая обработку до и после сети.

Критерий 5: показаны OOD-ошибки, `unknown`, пропуски и задержки; классическая ветка остаётся доступна. Критерий 6: подготовлен обзор «данные, модель и пределы применения» для интеграционной проверки. Наличие файла сохранённых весов не равно готовности к управлению. До физического использования отдельно проверяются датчики, исполнительная цепочка, независимые ограничения и остановка; календарное положение G3 не заменяет эти действия.

Самопроверка без конспекта

Объясни: 1) почему разделение по строкам завышает качество; 2) почему нельзя настраивать нормализацию на всех данных; 3) чем логины отличаются от вероятностей; 4) почему неподвижная стопа может не касаться пола; 5) почему контакт и отсутствие скольжения — разные цели; 6) как уверенное ложное подтверждение контакта повлияет на оценку состояния корпуса. Подкрепи ответы собственными графиками.

Ответ также: 7) чем калибровка отличается от классификации; 8) почему ROC может скрывать важность редких ошибок; 9) где возникает задержка накопления окна; 10) что обеспечивает `seed`; 11) почему экспорт требует повторной проверки; 12) какие измерения оправдывают GPU. Если ответ зависит от неизвестной характеристики робота, запиши её уточнение как требование следующего этапа.

Если не хватает времени

При нехватке времени исключи дополнительные архитектуры, затем сравнение MPS и GPU, затем практический опыт с квантованием, сохранив его теорию. Оставь CPU, небольшую сеть, простые признаки и данные, разделённые по группам. Сохрани базовые методы, проверку утечки, OOD, измерение полной задержки и воспроизводимость. В M19 передай формат вероятности и `unknown` с описанием ограничений. Оценка контакта для классической походки должна иметь проверенный путь без сети.

Литература: обучение на данных робота

Читай указанные фрагменты перед соответствующими задачами. Основные источники объясняют принципы, справочные помогают при реализации. Чтение входит в 75 часов. Источники могут быть на английском; объяснения и отчёт подготовь на русском.

Основное чтение

1. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor, An Introduction to Statistical Learning with Applications in Python, 2023. Главы 2, 4, 5: авторские материалы. Statistical Learning - ошибка и обобщение; Classification - логистическая регрессия; Resampling Methods - валидация и перекрёстная проверка. К задачам 1–5: обоснуй разделение эпизодов и выбор базового метода; все лабораторные работы выполнять не нужно.

2. Aston Zhang, Zachary Lipton, Mu Li, Alexander Smola, Dive into Deep Learning. §2.5 Automatic Differentiation; §3.7 Weight Decay; §5.1–5.3 (Multilayer Perceptrons; Implementation; Forward/Backward Propagation). К задачам 6–7: один небольшой MLP, градиент и обновление весов; упражнения выполняй на своих контактных признаках.

Справочник при непонятном результате

3. Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, 2016. Гл. 5 Machine Learning Basics; гл. 7 Regularization for Deep Learning. Найди объяснение сложности модели, переобучения и выбранной регуляризации для задачи 5. Гл. 8 Optimization for Training Deep Models - только выбранный оптимизатор задачи 7, без чтения всей главы.

Документация, нужная в эксперименте

PyTorch: Learn the Basics - Tensors, DataLoaders, Autograd, Optimization, Save/Load для задач 6–7. scikit-learn: Common pitfalls - Data leakage и preprocessing для задач 1–3; настройка преобразований только на обучающей выборке.

scikit-learn: Probability calibration - Calibration curves и Calibrating a classifier для задачи 9. На своей матрице ошибок объясни FP и FN, а также выбранный порог; дополни долю верных ответов разбором ошибок.

Сначала прочитай основной источник по теме, затем проверь понимание на задаче. К третьей книге обращай за объяснением конкретного затруднения; читать три учебника целиком в этом месяце не требуется. Измерения CPU и условия применения ускорителя описаны на странице подготовки.

Фундаментальные понятия M18

Признак (feature). Измеренная или вычисленная величина, доступная в момент решения; будущие значения недопустимы.

Метка (label). Целевой ответ для обучения; здесь обозначает контакт по принятому правилу, а не отсутствие скольжения.

Обучающая выборка (train). Данные для настройки параметров модели и преобразований признаков; только на них обучают эти параметры.

Валидационная выборка (validation). Отдельные данные для выбора гиперпараметров и порога; используется до итоговой проверки.

Тестовая выборка (test). Отдельные эпизоды для итоговой оценки; настройка по ним нарушает независимость проверки.

Утечка данных (data leakage). Недоступная при применении информация в обучении или оценке; например, соседние кадры в разных выборках.

Стандартизация (standardization). Вычитание среднего и деление на масштаб; статистики находят на обучающей выборке и фиксируют.

Логистическая регрессия. Сигмоида линейной комбинации признаков, задающая вероятность класса; здесь — бинарного контакта.

Функция потерь (loss). Мера расхождения прогноза и метки, уменьшаемая обучением; не равна полезности модели в работе.

Градиент (gradient). Вектор производных потерь по параметрам; задаёт направление локального роста для обновления весов.

Обратное распространение (backpropagation). Расчёт градиентов по цепному правилу; веса отдельным шагом меняет оптимизатор.

Регуляризация (regularization). Ограничение или штраф против подгонки особенностей обучающих данных; силу выбирают по валидации.

Переобучение (overfitting). Улучшение на обучающих данных за счёт их особенностей с ухудшением на новых эпизодах того же распределения.

Точность предсказаний (precision). Доля истинных контактов среди предсказанных: $TP/(TP+FP)$; отражает ложные подтверждения.

Полнота (recall). Доля найденных контактов среди всех положительных меток: $TP/(TP+FN)$; отражает пропуски при выбранном пороге.

Калибровка вероятностей. Согласование вероятностей с частотами на независимых данных; точная классификация её не гарантирует.

Порог решения (threshold). Граница выбора класса или отказа от ответа; задаётся по цене ошибок до итогового теста.

Сдвиг распределения (distribution shift). Изменение условий относительно обучения; новые материалы могут снизить качество при прежней уверенности.

Абляция (ablation). Исключение признака или компонента при неизменном протоколе, чтобы оценить его вклад и возможную утечку.

Задержка прогноза (inference latency). Время предсказания; полная задержка включает также подготовку, передачу и последующую обработку.

Что купить и установить к M18

Используй имеющееся оборудование

Используй MacBook Pro M4, прежние синхронизированные записи ноги и IMU, а также симулятор. Обучение и измерения времени выполняй прежде всего на CPU в macOS ARM. При необходимости экспортируй записи ROS из Ubuntu 24.04 ARM64 в UTM; сохрани episode_id, время, единицы и разделение признаков и эталонных данных.

Обязательные покупки не нужны

Для обязательного классификатора не нужны Jetson, NVIDIA GPU, новая камера и новый датчик контакта. Сначала измерь весь путь обработки данных небольшого MLP и логистической регрессии. Разметку контакта получай по принятому протоколу симулятора и храни отдельно от входных признаков модели.

Установить сейчас: отдельная Python-среда

В уже используемом uv/venv на macOS ARM добавь torch, scikit-learn и JupyterLab, если их ещё нет; NumPy/Matplotlib/pytest переиспользуй. Версию Python выбери по совместимости закрепляемых пакетов, сохрани lock и manifest. Инструкции: [PyTorch Start Locally](#), [scikit-learn install](#), [Jupyter install](#).

Выбирай выполнение на CPU; CUDA для Mac M4 не устанавливается. pandas и pyarrow добавляй только при необходимости чтения входных таблиц; для CSV достаточно принятого загрузчика. Зависимости обучения не устанавливай в системный Python ROS. Передавай данные между средами файлами согласованного формата.

Проверка установки и воспроизводимости

Импортируй torch и sklearn, проверь работу тензора на CPU, выполни прямой и обратный проход небольшой сети и один шаг оптимизатора. Обучи LogisticRegression на малой обучающей выборке, сохрани и повторно загрузи модель. После перезапуска вычислительной тетради воспроизведи сохранённое [разбиение по эпизодам](#) и [метрики классификации](#).

Зафиксируй значения seed, версии, число потоков и выбранное устройство; повтори короткий запуск по [PyTorch Reproducibility](#). Для задачи 12 измерь прогретые повторы и холодный старт, включая признаки и последующую обработку: [PyTorch Benchmark](#).

Опционально после измерения

Встроенный [MPS](#) можно сравнить с CPU после проверки доступности операций; CPU-результат сохраняется. Покупка ускорителя рассматривается только при доказанном вычислительном ограничении и отдельном решении G3. G3 27.11–10.12.2028 впереди, M18 его не закрывает.

Установка и проверки занимают время, уже отведённое в исходных 75 часах на настройку. Результат месяца — модель и измерения по записям или в симуляции. Классический алгоритм должен работать независимо от машинного обучения.

Основы движения: модель по данным

Обязательный разбор: чему регрессия учится о суставе ноги или пальца

От физического закона к обучаемым параметрам

Перед задачами 4 и 7 объясни различие неизвестного параметра и неизвестного состояния. Рассмотрим вращательный сустав в вертикальной плоскости без контакта: q — угол от нижней вертикали в рад, $\dot{q}$ — скорость в рад/с, $\ddot{q}$ — ускорение в рад/с². Положительный момент τ направлен в сторону роста q . Геометрия и направление оси уже проверены.

$I \cdot \ddot{q} + b \cdot \dot{q} + k \cdot \sin(q) = \tau$. I — момент инерции относительно оси, кг·м²; b - коэффициент вязкого трения, Н·м·с/рад; $k = m \cdot g \cdot l_c$ - известный коэффициент тяжести, Н·м; m - масса звена в кг, $g = 9.81$ м/с², l_c - расстояние от оси до его СОМ в м. Предполагаем жёсткое звено, постоянные I, b и отсутствие люфта. τ берём из учебной симуляции; заданный угол сервопривода и ток питания не являются измеренным суставным моментом.

Перенеси известный член момента силы тяжести: $y = \tau - k \cdot \sin(q)$, $\phi = (\ddot{q}, \dot{q})$, $\theta = (I, b)$.

Получим $y = \phi \cdot \theta$: движение нелинейно по q , но регрессия линейна по неизвестным I, b .

Минимизируй сумму квадратов невязок в Н²·м². Условия $I > 0$ и $b \geq 0$ выражают положительность кинетической энергии и рассеяние мощности $b \cdot \dot{q}^2$; малая ошибка сама по себе их не обеспечивает.

Малый расчёт: параметры и новая поза

Учебное $k = 0.20$ Н·м. Два независимых снимка при $q = 0$: $(\dot{q}, \ddot{q}, \tau) = (1, 2, 0.14)$ и $(2, -1, 0.03)$, в указанных SI-единицах. Получаем $2I + b = 0.14$ и $-I + 2b = 0.03$, откуда **$I = 0.05$ кг·м², $b = 0.04$**

Н·м·с/рад. Для отдельного снимка $q = \pi/6$, $\dot{q} = 0.5$, $\ddot{q} = -2$ прогноз $\tau = -0.10 + 0.02 + 0.10 = 0.02$ Н·м.

Два упражнения с базовой моделью

А. В существующей вычислительной тетради реши эту систему средствами NumPy, сохрани `id_check.csv` с входами, единицами и прогнозом для третьей строки. Проверь ответ подстановкой и баланс мощностей: во второй строке входная мощность $0.03 \times 2 = 0.06$ Вт равна скорости изменения кинетической энергии -0.10 Вт плюс потерям трения 0.16 Вт. Это проверка вычисления на известных данных; обобщение проверяется на отдельных эпизодах, как в задачах 1–5.

Б. Задай $\ddot{q} = 2 \cdot \dot{q}$ во всех строках. Матрица признаков имеет ранг 1: можно определить лишь $2I + b$, а не оба параметра по отдельности. Дополнительные одинаковые движения этого не исправят. В записи свободного движения проверь синхронизацию τ и q ; дифференцирование усиливает шум. Для текущей оценки используй только уже поступившие измерения. Контактные силы и эталон симулятора остаются в разметке и проверке; правила M18 сохраняются.

Часы: 1 ч из занятия 3–4 о регрессии и функции потерь и 1 ч из первой лаборатории 11–20, отведённый на анализ базового метода. Физический пример заменяет общий пример регрессии. Основной классификатор, разделение данных, OOD и измерения времени сохраняются. Итого прежние 35+40=75 ч.

Читать выборочно: Russ Tedrake, [System Identification](#): «Estimating inertial parameters (and friction)», «Lumped parameters for the simple pendulum», «Equation error vs simulation error». Объясни, почему малая невязка уравнения ещё не гарантирует точного прогноза на длительном интервале.